

P4-Days: Programmable Data-Plane Execution in a Multicore Discrete-Event Network Simulator

Taejoon Lee

School of Electrical Engineering and Computer Science

University of Ottawa, Ottawa, Canada

Email: tlee096@uottawa.ca

Abstract—Packet-level simulation of P4-programmable networks is expensive because a software P4 pipeline must be interpreted for every packet at every programmable hop. Existing integration through P4sim provides realistic BMv2 execution inside ns-3, but ns-3 uses a sequential event core. Days is a recent multicore discrete-event network simulator whose lightweight packet representation intentionally carries no serialized protocol bytes. Integrating BMv2 with Days therefore requires more than a direct wrapper. BMv2 requires packet bytes and persistent per-switch state, whereas Days must retain control of time, routing, links, queueing, and parallel execution. We present *P4-Days*, an extension that embeds one BMv2 instance in each programmable Days switch. P4-Days lazily synthesizes protocol headers, executes P4 ingress and egress on opposite sides of the Days scheduler, and isolates mutable BMv2 state at the switch level. Across 16 cross-simulator scenarios, P4-Days matches ns-3+p4sim whenever the two network models define the same outcome. P4-Days is $1.82\text{--}1.97\times$ faster with stock BMv2 and $1.94\text{--}2.52\times$ faster with logging-free BMv2. On a 1,280-switch fat-tree, P4-Days reaches a $5.19\times$ speedup on 16 workers, whereas unmodified Days reaches $3.96\times$. These results show that programmable packet processing can be integrated without destroying Days’ actor-level parallelism.

Index Terms—P4, BMv2, discrete-event simulation, multicore simulation, network simulation, programmable data plane

I. INTRODUCTION

P4 makes the data plane programmable [1], but evaluating a P4 program at datacenter scale remains costly. A packet-level simulator must execute a software representation of the switch pipeline once per packet and per hop. A large experiment can therefore require hundreds of millions of pipeline invocations. P4sim addresses fidelity by integrating BMv2, the reference P4 software switch, into ns-3 [2], [3]. This approach allows compiled P4 programs to execute inside the simulator. Its performance, however, remains bounded by ns-3’s sequential event core.

Days was recently proposed as a process-based packet-level discrete-event simulator implemented in Rust [4]. Network elements are actors that exchange messages and can be scheduled across multiple workers. This architecture creates an opportunity for programmable data-plane simulation. The P4 pipeline at one switch does not need to share mutable data-plane state with another switch. The additional packet-processing work can therefore be exposed as actor-level parallelism.

Integrating BMv2 into Days is not a mechanical task. A Days packet is a compact metadata structure that contains no Ethernet, IP, or transport bytes. BMv2 is a byte-oriented

interpreter whose parser, match-action stages, registers, and deparser operate on a packet-header vector. Days also pre-computes forwarding per flow and owns its queueing and serialization model. In contrast, the v1model architecture places a traffic manager between ingress and egress. BMv2 was also not designed as a library for a Rust actor runtime. Process-global initialization and per-switch state ownership therefore require explicit handling.

This paper presents *P4-Days*, a Days extension that executes unmodified BMv2-compatible P4 programs while retaining Days as the network simulator. Figure 1 summarizes the architecture. The paper makes four contributions. First, we introduce a packet-representation bridge that synthesizes Ethernet, IP, and transport headers only when a packet first reaches a programmable switch. The bridge preserves P4-written bytes across subsequent hops. Second, we split P4 execution around the Days scheduler. BMv2 ingress executes before enqueue, whereas BMv2 egress executes at dequeue so that queueing metadata and packet-size changes correspond to the simulated queue and link. Third, we assign each programmable switch an independent BMv2 instance and validate that this ownership model preserves actor-level parallel execution without a global BMv2 lock. Fourth, we evaluate P4-Days through cross-simulator validation against ns-3+p4sim, single-worker runtime measurements, per-hop cost decomposition, and multicore scaling experiments.

II. RELATED WORK

P4-Days lies at the intersection of programmable-data-plane simulation and high-performance discrete-event simulation. P4sim is the closest functional reference because it embeds BMv2 into ns-3 so that compiled P4 programs execute inside simulated switches while ns-3 provides topology, traffic, and timing [2]. NS4 is an earlier ns-3 extension for P4-driven network simulation. It demonstrated the value of defining programmable forwarding behavior instead of reproducing that behavior manually inside a simulator [5]. P4-Days follows the same broad objective, but it targets a different execution substrate. Days is a multicore actor-based simulator whose packets do not contain serialized headers.

PFPSim approaches programmable-data-plane simulation from the device side. It generates host-compiled models of forwarding-plane architectures for pre-silicon design-space exploration and can execute applications written in P4 [6]. Its

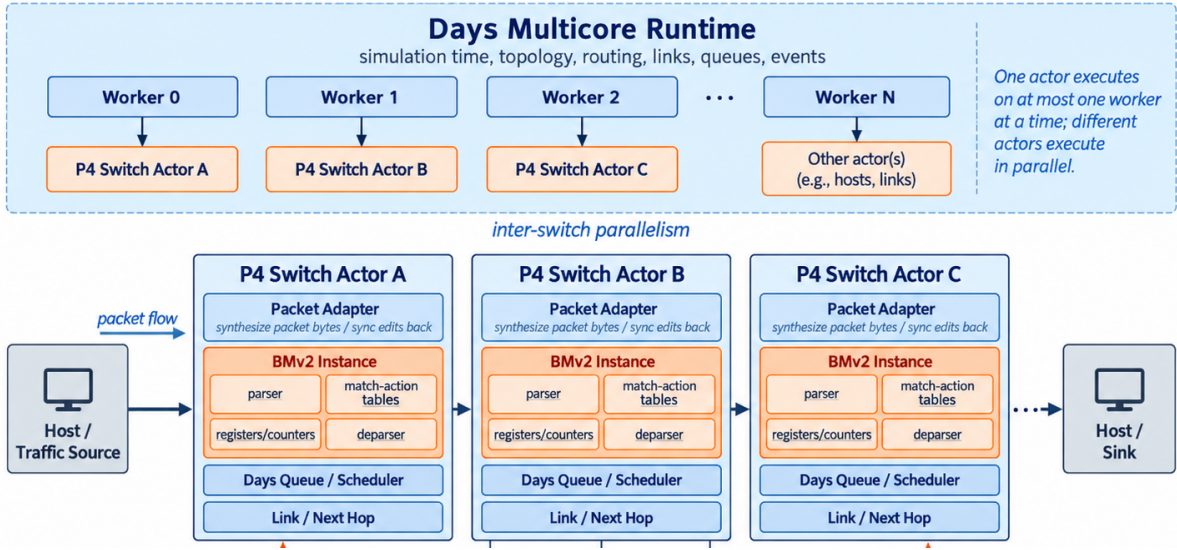


Fig. 1. P4-Days architecture. Days retains control of time, actor scheduling, routing, links, and queues. Each programmable switch actor owns an independent BMv2 instance and can execute in parallel with other actors.

primary goal is detailed forwarding-device architecture analysis rather than end-to-end packet-level network simulation. P4-Days instead leaves topology, queues, links, routing, and event scheduling in Days and uses BMv2 only as the programmable packet-processing engine inside each switch.

Recent work has improved packet-level simulation scalability through several different architectures. DONS reorganizes discrete-event simulation around data-oriented execution and automatic parallelization, and it demonstrates efficient use of multicore and distributed resources [7]. Days instead uses actors on the nexosim runtime and is the simulator extended in this work [4]. P4-Days addresses a complementary question. It asks whether a stateful and byte-oriented programmable switch can be embedded in a multicore discrete-event simulator without introducing global serialization or replacing the surrounding network model.

SimBricks uses a different approach in which existing simulators execute as separate processes and synchronize across explicit component boundaries [8]. This co-simulation model is useful for heterogeneous end-to-end systems that combine host, device, and network simulators. P4-Days instead keeps a single network simulator and executes BMv2 inside individual switch actors. Inter-switch parallelism therefore comes from Days itself, while BMv2 remains a per-switch packet-processing engine.

ns-3 remains a widely used network-simulation ecosystem [9], and BMv2 remains the reference software switch for P4 development and testing [3]. P4-Days does not propose a new P4 target or a new general-purpose simulator. Its contribution is an integration architecture and an evaluation that show how programmable data-plane execution can coexist with metadata-oriented packets, simulator-owned queues, and multicore actor scheduling.

III. BACKGROUND AND DESIGN GOALS

A. Days and BMv2

Days models a network as actors on the nexosim runtime. A model is `Send` but not `Sync`. The runtime may move a model between workers, but at most one worker executes a given actor at a time. This ownership discipline serializes state within a switch while exposing parallelism across independent switches. Days also keeps packets intentionally lightweight. A packet stores fields such as size, flow and packet identifiers, priority, ECN state, and timestamps, but it does not store serialized headers or payload bytes.

BMv2 uses a different packet representation. It consumes a byte buffer, parses headers into a packet-header vector (PHV), applies match-action tables and stateful primitives, and deparses modified bytes [3]. These operations provide the behavior needed to run compiled P4 programs. They also require packet bytes and pipeline state that do not exist in the default Days packet representation.

B. Design Goals

The design follows four requirements. First, a simulation with P4 disabled must behave as an ordinary Days simulation. Second, P4 processing must not take ownership of simulated time, routing, queues, or links. Third, mutable P4 state must remain isolated at the switch level so that different switches can execute concurrently. Fourth, the P4 pipeline must observe queue state at the same logical point as a v1model traffic manager. P4 execution therefore cannot be attached as an unrelated post-processing step.

IV. DESIGN AND IMPLEMENTATION

A. Lazy Packet Synthesis and Persistent Bytes

P4-Days creates serialized protocol bytes only when they are needed. When a packet first reaches a P4-enabled switch,

P4-Days synthesizes Ethernet, IPv4, and TCP or UDP headers from Days metadata. The resulting bytes are stored in an optional P4 payload attached to the packet and remain attached at subsequent programmable hops. This approach avoids converting the entire simulator to a byte-oriented packet model. It also avoids payload allocation for packets that never enter a P4 switch.

The metadata-to-header mapping is centralized in one layer. Host identifiers are mapped to IPv4 and MAC addresses. The 32-bit `flow_id` is encoded across the L4 source and destination port pair. Priority occupies the upper DSCP bits, and ECN occupies the standard lower two bits of the IPv4 TOS byte. Reverse-direction packets, including TCP acknowledgments and DCQCN control packets, swap endpoints during synthesis. Before every P4 pipeline invocation, P4-Days synchronizes the current Days-side ECN and priority values back into the serialized representation. This synchronization prevents queue-side markings from being overwritten by stale packet bytes.

Days continues to select paths. Its forwarding decision, including ECMP selection, is installed into a P4 table as one exact entry per flow. P4-Days therefore targets programmable packet processing rather than programmable routing. A P4 program can rewrite headers, mark packets, meter traffic, update counters and registers, drop packets, and inspect queue state. It cannot redirect a flow to a next hop that differs from the path selected by Days.

B. Splitting the P4 Pipeline Around the Days Queue

The location of egress execution is a key design decision. In `v1model`, the traffic manager lies between ingress and egress. The corresponding component in Days is the per-port scheduler. P4-Days therefore executes the parser and ingress control in the P4 switch actor and records the selected output port. It then carries the live BMv2 packet into the Days scheduler. At dequeue, a P4 egress hook populates queueing metadata, including dequeue depth and dequeue time. The hook then executes egress and deparser before packet-size changes are committed to link serialization.

This ordering is necessary for meaningful queue telemetry. A queue-probe P4 program reports a maximum dequeue depth of 999 for a congested queue with capacity 1000, while it reports zero on an uncongested switch in the same run. The design also requires the BMv2 packet to remain live while it is queued. Scheduling disciplines that clone packets on enqueue are therefore unsupported in the current implementation. FIFO and DRR are supported.

C. BMv2 as a Synchronous Per-Switch Library

P4-Days does not start BMv2’s internal packet-processing threads or its Thrift control server. Instead, a small C++ shim subclasses `bm::Switch`, loads the compiled JSON through `init_objects()`, obtains the parser, ingress, egress, and deparser objects, and drives those objects synchronously from the actor runtime. P4 tables are populated directly in process.

Every simulated P4 switch owns a separate BMv2 instance. The Rust wrapper is `Send` but intentionally not `Sync`.

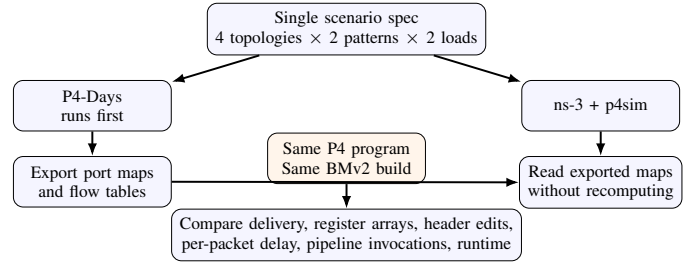


Fig. 2. Cross-simulator validation workflow. Days generates routing and port mappings once. The ns-3 side reuses those artifacts so that topology bookkeeping cannot masquerade as simulator disagreement.

We also tested concurrent libbm2 execution because BMv2 contains process-global facilities in addition to per-switch objects. A stress test with four switches showed that libbm2 can be driven concurrently after its lazily initialized global `EventLogger` is constructed before worker threads begin execution. Under this initialization order, Helgrind reported zero data races. Parallel execution also produced byte-identical output to sequential execution for all 4,000 test packets. The implementation detects unsupported replication primitives, including clone, recirculate, and multicast, in the compiled JSON and rejects them explicitly.

V. VALIDATION METHODOLOGY

The evaluation separates two questions. The first question is whether P4-Days preserves the intended network behavior. The second question is whether the BMv2 integration preserves the parallel performance that motivates the use of Days.

A. Internal Validation

Five staged tests exercise passthrough behavior, header editing, match-action forwarding, register persistence, and state isolation across two switches. The original Days tests also pass in the default build, the P4-only build, and the P4+BMv2 build. We additionally compare a simulation that traverses P4 switches with the same unmodified Days configuration. Sink output is identical, including under overload.

B. Cross-Simulator Scenarios

External validation compares P4-Days with ns-3+p4sim across 16 scenarios. The matrix contains four topologies, two traffic patterns, and two offered loads. The topologies are a single bottleneck, leaf-spine, three-tier Clos, and a $k = 4$ fat-tree. The traffic patterns are incast and permutation, and the offered loads are 0.6 and 1.5. All links operate at 1 Gb/s with 1 μ s propagation delay and 1042-byte wire frames. The workload uses open-loop UDP so that transport implementations do not confound the network-model comparison.

A single source description generates both simulators’ scenarios. Days executes first, assigns port numbers, computes ECMP paths, and exports the resulting mappings. The ns-3 side reads those artifacts rather than recreating them independently. Both simulators link against the same BMv2 build. We report stock BMv2 and a logging-free build separately because

TABLE I
CONTROLS USED FOR A LIKE-FOR-LIKE COMPARISON.

Control	Rationale
Same BMv2 build	This control prevents compile-time logging configuration from being mistaken for simulator overhead.
One generated scenario	This control prevents topology, port-number, and ECMP mismatches between independently authored configurations.
1042-byte wire frames	The ns-3 sender uses a 1000-byte UDP payload, whereas the Days size represents on-wire bytes.
Exported routing	Days assigns port indices and ECMP paths once, and ns-3 reads those assignments instead of recomputing them.
Open-loop traffic	This control avoids comparing unrelated TCP implementations while validating the network model.
Separate verification and timing programs	Register instrumentation is useful for correctness checks, but it adds about one microsecond per hop.

commonly enabled BMv2 debug macros perform expensive string construction even when no log message is emitted.

The verification P4 program records counts, byte totals, and post-pipeline TTL and TOS aggregates per flow and egress port. These registers test both forwarding behavior and header mutation. Runtime measurements use the same forwarding logic without the verification registers. This separation prevents register instrumentation from being counted as simulator overhead.

C. Experimental Environment and Fairness Controls

All measurements in this draft were collected on one Ubuntu 24.04 machine with 31 GB of RAM and sixteen usable workers. The cross-simulator experiments use one worker on the Days side because ns-3’s discrete-event core is sequential. Multicore behavior is evaluated separately. Each reported runtime is the median of the repeated runs specified by the experiment scripts.

Table I summarizes the controls required for a like-for-like comparison. These controls are part of the experimental methodology. Several earlier configurations produced plausible but incorrect conclusions when one or more controls were violated.

D. Lessons from Cross-Simulator Validation

The validation process exposed several issues that produced believable but incorrect results instead of explicit failures. An early experiment compared a 1000-byte Days packet with a 1000-byte ns-3 UDP payload. The mismatch made the Days link approximately 4% faster and changed the qualitative queueing regime at exactly 100% load. A p4sim queue-size parameter also behaved as a packet count at one queue even though its name suggested a byte count, while a second device queue retained a different default. In another case, a missing path separator prevented flow tables from loading, which caused every packet to take the P4 default-drop action without an explicit error. Synchronized source start times also created repeated same-timestamp ties at full buffers and amplified otherwise unimportant scheduler ordering differences.

These failures motivated the workflow in Fig. 2. One scenario description generates both simulator inputs. Packet

size is defined at the wire level. Routing and port mappings are exported rather than recreated. Validation also checks internal P4 state in addition to endpoint statistics. Endpoint percentiles alone are insufficient because two simulations can exhibit similar distributions while delivering different individual packets.

VI. EVALUATION

A. Cross-Simulator Fidelity

Register arrays are identical in 11 of the 16 scenarios, and total pipeline invocations are identical to the digit in 15 of the 16 scenarios. Every scenario whose busiest output port remains at or below $1.0\times$ offered load agrees exactly. Differences in per-flow allocation appear under persistent saturation. The first divergence occurs when an output queue is full and same-timestamp arrivals from different input ports compete for a newly available slot. Days and ns-3 use different event-ordering mechanisms, and neither simulator specifies a physical input-arbitration policy. In the same overloaded scenarios, unmodified Days and P4-Days produce byte-identical per-flow results. This observation indicates that the divergence comes from the underlying simulator models rather than from the BMv2 integration.

The distinction is important for interpreting validation results. Cross-simulator equality is meaningful when the modeled system defines a unique outcome. When input arbitration is unspecified, forcing numerical equality through parameter tuning would conceal a modeling difference rather than validate the implementation.

1) *Why Persistent Overload Diverges*: The pattern in Table II follows hotspot load more closely than declared source load. The mechanism becomes deterministic once a buffer remains full. Consider a departure at simulated time t that frees exactly one slot while packets from two different input ports also arrive at time t . The first arrival processed by the simulator occupies the slot. The winning flow can then become phase-aligned with the next departure, which creates a persistent winner and loser pattern. In one measured case, the losing flow delivered exactly 1,000 packets. Their sequence numbers were 0 through 999 with no gaps, which equals the output-buffer capacity, and that flow did not win another slot afterward.

A physical switch resolves simultaneous arrivals from different input ports through an input-arbitration mechanism such as round-robin or priority arbitration. Neither simulator in this comparison models such an arbiter. Days enqueues packets according to actor-message delivery order, while ns-3 follows event-queue order. The overloaded cases therefore do not represent two different answers to a fully specified physical model. For integration validation, the relevant checks are that P4-Days reproduces unmodified Days and that the boundary of cross-simulator agreement can be explained by the routed hotspot. Both conditions hold in these experiments.

B. Sequential Runtime Against ns-3+p4sim

Figure 3 reports wall-clock runtime ratios over all 16 scenarios. With stock BMv2 linked on both sides, P4-Days is $1.82\text{--}1.97\times$ faster, with a median of $1.90\times$. With the same

TABLE II
CROSS-SIMULATOR AGREEMENT OVER 16 SCENARIOS. “REG.” AND “HDR.” INDICATE EXACT REGISTER-ARRAY AND HEADER-EDIT AGREEMENT. HOTSPOT IS THE OFFERED LOAD AT THE BUSIEST OUTPUT PORT AFTER ECMP PLACEMENT.

Scenario	Topology	Pattern	Declared load	Hotspot	Delivered	Reg.	Hdr.	Delay spread (pkt times)
S1	bottleneck	incast	0.6	0.60	equal	yes	yes	0.18
S2	bottleneck	incast	1.5	1.50	equal	yes	yes	0.46
S3	bottleneck	perm.	0.6	0.60	equal	yes	yes	0.18
S4	bottleneck	perm.	1.5	1.50	equal	yes	yes	0.46
S5	leaf-spine	incast	0.6	0.60	equal	yes	yes	0.00
S6	leaf-spine	incast	1.5	1.50	equal	yes	yes	8.00
S7	leaf-spine	perm.	0.6	1.20	equal	yes	yes	1.35
S8	leaf-spine	perm.	1.5	3.00	equal	no	no	1.85
S9	3-tier Clos	incast	0.6	0.60	equal	yes	yes	1.07
S10	3-tier Clos	incast	1.5	1.50	equal	yes	yes	4.00
S11	3-tier Clos	perm.	0.6	1.20	equal	no	no	2.36
S12	3-tier Clos	perm.	1.5	3.00	differs	no	no	1000.08
S13	fat-tree	incast	0.6	0.60	equal	yes	yes	0.01
S14	fat-tree	incast	1.5	1.50	equal	no	no	13.00
S15	fat-tree	perm.	0.6	1.20	equal	yes	yes	1.10
S16	fat-tree	perm.	1.5	3.00	off by 1	no	no	250.23

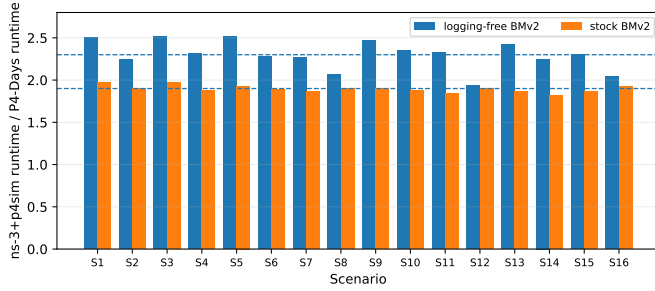


Fig. 3. Single-worker runtime advantage. Each bar reports $T_{ns-3+p4sim}/T_{P4-Days}$ with the same BMv2 build on both sides.

logging-free BMv2 library linked on both sides, P4-Days is $1.94\text{--}2.52\times$ faster, with a median of $2.30\times$. The ratio does not show a consistent trend with topology size. The lower ratios occur in scenarios dominated by full queues rather than in scenarios with larger graphs.

C. Cost of One P4 Packet Hop

On a logging-free build without congestion, P4-Days adds about $1.47\text{ }\mu\text{s}$ per pipeline invocation. Figure 4 decomposes this cost. BMv2 parser, ingress, and deparser execution account for about $0.69\text{ }\mu\text{s}$. BMv2 packet and PHV allocation and release account for roughly $0.25\text{ }\mu\text{s}$. Metadata reset accounts for about $0.24\text{ }\mu\text{s}$. The integration-specific Rust synthesis, reverse mapping, and FFI account for approximately $0.24\text{ }\mu\text{s}$. Most of the remaining cost therefore comes from the execution substrate needed to run BMv2 correctly rather than from the integration glue itself.

Moving egress to dequeue provides real `queueing_metadata`, but it also increases runtime. The increase is 12–16% in uncongested scenarios and reaches 55% when two saturated queues retain many live packets. Stage profiling shows that egress computation itself does not become expensive. The dominant costs instead come from memory locality and PHV-pool pressure because BMv2 packets remain live while they reside in the queue.

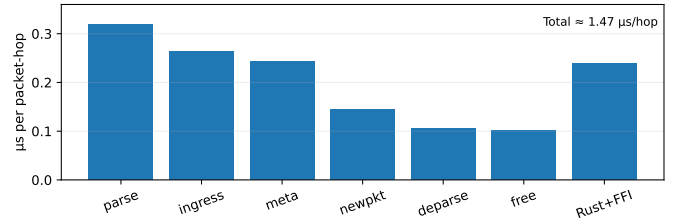


Fig. 4. Per-packet-hop cost decomposition for the logging-free build.

TABLE III
COST OF PLACING EGRESS AT DEQUEUE, WHICH MAKES `QUEUEING_METADATA` CORRESPOND TO THE SIMULATED QUEUE.

Mean queued packets	Representative scenarios	Runtime increase
0	S1, S3, S5, S13	12–16%
209	S7, S11	21–24%
975	S2, S4, S6, S8, S10, S14	23–37%
1854	S12	55%

The cost in Table III correlates strongly with queue residency. The measured Spearman correlation is $\rho = 0.952$ over the fourteen scenarios for which this experiment was run. Profiling two otherwise identical cases explains the trend. Parser and ingress costs remain nearly unchanged. The egress control also remains unchanged because the evaluation program has an empty egress body. The increase instead appears in packet allocation, deparse, and release after a long queue residence. Keeping many live PHVs enlarges the pool and causes the first memory accesses after dequeue to miss cache more often. This result identifies a concrete optimization target. A future implementation could release the PHV at enqueue, carry only deparsed bytes through the queue, and reconstruct the state needed for egress at dequeue.

D. Multicore Scaling

The central scalability question is whether the BMv2 integration serializes Days. We evaluate fat-trees with $k \in \{8, 16, 32\}$ using 1, 2, 4, 8, and 16 workers. The workload

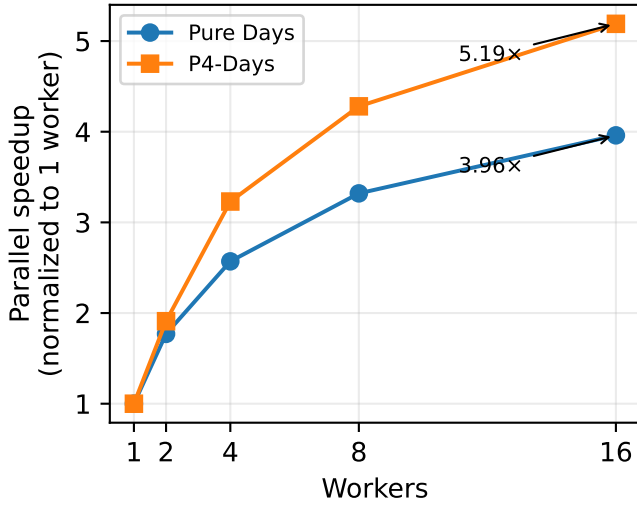


Fig. 5. Parallel speedup on the largest evaluated topology ($k = 32$, 1,280 switches). Each curve is normalized to the one-worker runtime of the same variant. P4-Days reaches $5.19\times$ at 16 workers, whereas pure Days reaches $3.96\times$. The figure reports scaling rather than absolute runtime. Full $k = 8, 16, 32$ results appear in Table IV.

remains near three million packets at each topology size. Setup time is excluded from the parallel speedup metric because topology construction and the creation of one BMv2 instance per switch occur during serial initialization. We report initialization costs separately in Section VI-E so that both steady-state parallelism and end-to-end deployment cost remain visible.

Figure 5 normalizes each curve to the corresponding variant’s one-worker simulation time. On $k = 32$, which contains 1,280 switches, unmodified Days improves from 26.325 s to 6.653 s and reaches a $3.96\times$ speedup. P4-Days improves from 101.552 s to 19.552 s and reaches a $5.19\times$ speedup. P4-Days is not faster than pure Days in absolute runtime. Instead, its additional per-switch P4 work parallelizes effectively. The relative P4 overhead falls from $3.86\times$ on one worker to $2.94\times$ on 16 workers. A global BMv2 lock would be expected to produce the opposite trend.

The $k = 8$ topology flattens after eight workers because it contains only 80 switches, which leaves five switches per worker at 16 workers. P4-Days continues to scale through 16 workers at $k = 16$ and $k = 32$. Memory becomes the more immediate scale limit because one BMv2 instance consumes approximately 10.5 MB.

E. Memory and Initialization Scaling

Compute scaling is not the only limit of the current design. The one-instance-per-switch model intentionally trades memory for simple ownership and parallel safety. Table V shows a stable cost of roughly 10.5 MB per programmable switch across a $16\times$ range in switch count. The $k = 32$ experiment therefore consumes 13.6 GB for 1,280 P4 switches on a 31 GB machine. Extrapolating the same design to $k = 64$, which would contain 5,120 switches, gives an estimated requirement

of roughly 54 GB. That configuration does not fit on the evaluation machine. The resulting limit is a memory ceiling rather than a parallel execution ceiling.

Initialization cost is also substantial at large switch counts. At $k = 32$, the serial setup for pure Days is about 97.7 s, compared with 26.325 s of one-worker simulation time. P4-Days adds approximately 4.07 ms of initialization per programmable switch. Across 1,280 switches, this corresponds to about 5.21 s of additional setup and places the P4-Days setup time near 103 s. These costs matter for end-to-end experiment turnaround, but they measure topology construction, routing, and switch initialization rather than parallel event execution. We therefore report setup time separately from simulation time. For example, including the 97.7 s serial setup would reduce the apparent pure-Days 16-worker speedup from $3.96\times$ to approximately $1.19\times$, even though the simulation phase itself scales by $3.96\times$.

A future backend could reduce both the memory floor and initialization cost by sharing immutable P4 program structures across switches while retaining mutable registers, tables, and PHV state per switch. Such an optimization would preserve the Days actor interface while reducing the cost of the current ownership model.

VII. REPRODUCIBILITY AND ARTIFACT STRUCTURE

The implementation uses feature gates so that the default Days build contains no P4 code and no C++ dependency. A pure-Rust p4 layer contains packet mapping, port mapping, and switch plumbing. The `p4_bmv2` feature adds the C++ backend. The repository includes the P4 programs used for functional tests, the 16-scenario cross-simulator artifact, multicore scaling scripts, and per-stage profiling tools. The main workflows are script-driven. The scenario generator emits both Days and ns-3 configurations, Days exports selected port and flow mappings, and the analysis scripts return a nonzero exit status when expected equivalence checks fail.

The artifact exposes three forms of diagnostic output. It records endpoint arrivals, per-hop P4 events, and final BMv2 register dumps. Measurement runs disable per-packet logs because logging adds 11–13% runtime overhead. Debug runs can additionally enable BMv2 tracing. The artifact also retains both a stock BMv2 installation and a separate logging-free build. Performance comparisons can therefore link both simulators against identical BMv2 libraries. This separation is necessary because compile-time BMv2 logging macros account for a large fraction of pipeline cost even when no messages are emitted.

The public artifact is intended to make the paper’s two main claims independently testable. The first claim is that P4-Days preserves Days behavior and agrees with ns-3+p4sim within the stated model boundary. The second claim is that the integration retains multicore scaling. A subset of the cross-simulator scenarios can be rerun as a smoke test for the first claim. Reproducing the $k = 32$ scaling result requires approximately 14 GB of RAM. The artifact reports simulation

TABLE IV
MULTICORE SCALING RAW SIMULATION TIMES IN SECONDS AND CORRESPONDING SPEEDUPS. EACH TOPOLOGY CARRIES APPROXIMATELY THREE MILLION PACKETS. SETUP TIME IS EXCLUDED AND IS REPORTED SEPARATELY.

Variant	k	Runtime (s) at workers				Speedup at workers					
		1	2	4	8	16	1	2	4	8	16
Pure Days	8	1.612	1.294	0.981	0.828	0.805	1.00	1.25	1.64	1.95	2.00
P4-Days	8	8.472	4.849	2.970	2.807	2.998	1.00	1.75	2.85	3.02	2.83
Pure Days	16	11.407	6.394	3.966	2.789	2.297	1.00	1.78	2.88	4.09	4.97
P4-Days	16	56.068	30.171	17.091	12.993	11.210	1.00	1.86	3.28	4.32	5.00
Pure Days	32	26.325	14.903	10.234	7.935	6.653	1.00	1.77	2.57	3.32	3.96
P4-Days	32	101.552	53.065	31.486	23.711	19.552	1.00	1.91	3.23	4.28	5.19

TABLE V
MEMORY COST OF THE ONE-BMV2-INSTANCE-PER-SWITCH DESIGN.

Fat-tree	Switches	Pure Days RSS	P4-Days RSS
$k = 8$	80	14 MB	879 MB
$k = 16$	320	56 MB	3,477 MB
$k = 32$	1,280	272 MB	13,591 MB

time separately from serial setup so that the two costs can be examined independently.

VIII. DISCUSSION AND LIMITATIONS

P4-Days intentionally preserves Days network semantics instead of turning Days into a general P4 hardware model. A P4 program cannot select a next hop that differs from the routing decision made by Days. Clone, recirculate, and multicast are rejected because the BMv2 replication engine is not part of the synchronous integration. P4-enabled schedulers are currently restricted to disciplines that do not clone the live BMv2 packet on enqueue. Link rates are uniform because of a current Days configuration limitation. The cross-simulator agreement experiments use one worker on the Days side so that model fidelity can be evaluated separately from parallel event-ordering differences.

The evaluation also exposes a practical performance trade-off. Keeping a live BMv2 packet across a simulated queue is the simplest way to preserve pipeline state and provide accurate dequeue metadata. The same choice increases memory pressure and reduces locality when queue residence is long. A future design could deparse and release the PHV at enqueue, carry only packet bytes through the queue, and reconstruct the state required for egress at dequeue. This alternative would increase implementation complexity but could substantially reduce the live-state footprint.

The current cross-simulator suite validates one forwarding program and a verification variant that adds counting and TTL and TOS checks. A separate queue probe validates queue metadata against the Days scheduler. Meters, hashing, and more elaborate stateful programs have not yet been cross-validated against p4sim. The implementation supports these classes of operations only to the extent exercised by the current integration and test suite. Broader P4-program coverage is therefore an important direction for future validation.

IX. CONCLUSION

P4-Days embeds BMv2 P4 execution into Days without replacing the simulator’s topology, routing, timing, queueing, or actor runtime. The integration resolves the representation mismatch between metadata-only Days packets and byte-oriented BMv2 processing. It also places ingress and egress on opposite sides of the simulated queue and isolates mutable BMv2 state at the switch level. Cross-simulator experiments show close agreement with ns-3+p4sim whenever both network models define a unique outcome. P4-Days is also consistently faster in the one-worker comparison. More importantly, the added P4 processing preserves multicore scaling. On the 1,280-switch topology, P4-Days reaches a $5.19\times$ speedup on 16 workers. The current design therefore shows that programmable data-plane execution is compatible with process-based multicore network simulation. Its principal scaling constraint is the memory and initialization cost of maintaining one BMv2 instance per programmable switch rather than serialized packet execution.

REFERENCES

- [1] P. Bosshart *et al.*, “P4: Programming protocol-independent packet processors,” *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 3, pp. 87–95, 2014.
- [2] M. Ma and G. T. Nguyen, “P4sim: Simulating programmable switches in ns-3,” in *Proceedings of the International Conference on ns-3 (ICNS3)*, 2025, pp. 107–115.
- [3] P4 Language Consortium, “Behavioral model (bmv2): The reference p4 software switch,” GitHub repository. [Online]. Available: <https://github.com/p4lang/behavioral-model>
- [4] B. Li, “Days: Discrete-event network simulation on steroids,” in *Proceedings of the IEEE Conference on Computer Communications (INFOCOM)*, 2026, pp. 1–10.
- [5] J. Bai, J. Bi, P. Kuang, C. Fan, Y. Zhou, and C. Zhang, “Ns4: Enabling programmable data plane simulation,” in *Proceedings of the ACM Symposium on SDN Research (SOSR)*, 2018.
- [6] S. Abdi, U. Aftab, G. Bailey, B. Boughzala, F. Dewal, S. Parsazad, and E. Tremblay, “P4psim: A programmable forwarding plane simulator,” in *Proceedings of the ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*, 2016, pp. 55–60.
- [7] K. Gao, L. Chen, D. Li, V. Liu, X. Wang, R. Zhang, and L. Lu, “Dons: Fast and affordable discrete event network simulation with automatic parallelization,” in *Proceedings of the ACM SIGCOMM Conference*, 2023, pp. 167–181.
- [8] H. Li, J. Li, and A. Kaufmann, “Simbricks: End-to-end network system evaluation with modular simulation,” in *Proceedings of the ACM SIGCOMM Conference*, 2022.
- [9] G. F. Riley and T. R. Henderson, “The ns-3 network simulator,” in *Modeling and Tools for Network Simulation*. Springer, 2010, pp. 15–34.